Automated Bootstrapping Placement for Encrypted Inference

CS5491 Artificial Intelligence Project Proposal

Member 1: [Confirmed English name] | Student ID: [ID]
Member 2: [Confirmed English name] | Student ID: [ID]
Member 3: [Confirmed English name] | Student ID: [ID]
Contact email: [Email]

Problem and motivation

We will optimise bootstrapping placement for the supplied MLP and attention blocks used in encrypted inference. Multiplicative operations consume ciphertext levels, while expensive bootstrapping restores them. Residual connections and shared tensors make placement a graph problem: refreshing a producer once can benefit several consumers. Our objective is to minimise the handout's total estimated inference latency while preserving every original operator and satisfying all level constraints.

Method

We formulate integer input and output levels for each operator, and a refresh choice and delivered level for each edge. These decisions represent no insertion, Bootstrap1, Bootstrap2, Modswitch, or a refresh followed by Modswitch. Activation variables charge each shared refresh once. Bootstrap2 is restricted to tensors of 8 or 16 ciphertexts and is charged per pair. We implement the model and DAG reconstruction in Python, using Google's OR-Tools CP-SAT as the search engine. Costs are scaled by 10,000 to retain the supplied decimal precision. An independent checker verifies graph preservation, legal chains, levels and shared costs before comparison with the instructor's scorer. Exhaustive tiny examples provide a separate correctness check.

Evaluation

The benchmark comprises 30 instances: MLP and attention, each at input levels 0 through 14 with maximum level 14. We will compare every solution against the instructor's fixed placement at the same input level. We will report estimated latency and its percentage reduction, ciphertext-based bootstrap counts, depth, budget and utilisation. Saved DAGs and placement diagrams will expose all inserted operations and edge levels. Solver time, CPU memory, status and objective bounds will be recorded separately from fitted inference latency. Feasible solutions will be distinguished from solutions proved optimal for the model.

Feasibility and delivery

We have calibrated all 30 teacher baselines. The current prototype has independently validated improvements for all 15 MLP inputs and attention inputs 0, 9 and 14. Preliminary latency reductions are 7.32-22.11% for MLP and 14.68-22.44% for these attention cases. We will complete attention coverage, freeze the experiment configuration and reproduce the results in a clean CPU environment. The milestone will provide code, evaluation and preliminary analysis by 19 November; the final report and reproducible repository will follow by 3 December. No model training, GPU or LLM API is required by this optimisation method.